

EC-241 OOP

LECTURE 3

1

Overloaded Constructors

```

class GradeBook
{
public:
    GradeBook()                //default ctor
    {   courseName="OOP";   }
    GradeBook(string name)     //overloaded ctor
    {
        setCourseName(name); //call set function to initialize courseName
    }
    void setCourseName(string name)
    {   courseName=name; //store course name in the object   }
    string getCourseName()
    {   return courseName; //return the object's course name   }
    void displayMessage ()
    { cout<<"Welcome to the Grade Book for\n" << getCourseName() << "!" <<endl;
      }
private:
    string courseName;         //course name for this GradeBook
}; //end of class GradeBook

```

2

Overloaded Constructors

```
int main()
{
    GradeBook book1;
    GradeBook book2("Data Structures");
    cout<<book1.getCourseName();
    cout<<book2.getCourseName();
    return 0;
}
```

Note: In C++ any function can be overloaded.

Overloaded functions are defined as functions with same name but different in either the number of arguments, or the types of arguments (or both)

3

CONSTRUCTOR TYPES LEARNT SO FAR

- **Default constructor**
 - No arguments
 - May be implicit or explicit
- **Overloaded constructor**
 - Has arguments

Remember:

An implicit default constructor is built into the program automatically by the compiler if you don't provide one.

4

Class Work

```
class Rectangle
{
    //data members ?
    /*member functions:
        default ctor
        overloaded ctor
        setvalues
        getvalues
        area
        perimeter
    */
};
```

```
void main()
{
    Rectangle r1;
    r1.setvalues(3,7);
    Rectangle r2(2,6);
    cout<<r1.area();
    cout<<r2.area();
    cout<<r1.perimeter();
    cout<<r2.perimeter();
}
```

5

ASSIGNMENT NO. 1

- TEXT BOOK (DEITEL, 6TH ED.) EXERCISES 3.12, 3.13
- DUE NEXT WEEK (SAME DAY)
- GROUP OF 2 ALLOWED
- WHAT TO SUBMIT:
 - PROGRAM SOURCE CODE (PRINTED HARD COPY)
 - OUTPUT OF TEST PROGRAM

6

Copy Constructors

- It is a member function which initializes an object using another object of the same class.
- Invoked (implicitly) in following situations:
`ClassType obj1;`
`ClassType obj2 = obj1; // Note: This is NOT assignment`
- Other situations that invoke copy ctor:
 - An object is passed **by value** to a function
 - An object is **returned as the value** of a function, as in `return myGradeBook;`
- Takes one argument, a reference to class type:
`ClassType(const ClassType & T);`

7

Default Copy Constructor

- In the absence of a programmer-provided copy constructor, the C++ compiler builds a default copy constructor for each class which performs a **member-wise copy** between objects.
- i.e. members of one object are copied to corresponding members of the other object
- Default copy constructors **work fine unless the class contains pointer data members (more on this later)**

8

Constructors with Default Arguments

```
//Time.h
class Time
{
    private:
        int hour, minute, second;
    public:
        Time(int=0, int=0, int=0);    //default ctor
        ...
        ...
        ...
};
```

9

Constructors with Default Arguments

```
//Time.cpp
#include "Time.h"
Time::Time(int hr, int min, int sec)
{
    hour=hr;
    minute=min;
    second=sec;
}
...
...
...
```

10

Constructors with Default Arguments

```
//TestTime.cpp
...
#include "Time.h"
void main()
{
    Time t1; // 0 0 0
    Time t2(2); // 2 0 0
    Time t3(21, 34); // 21 34 0
    Time t4(12, 25, 42); // 12 25 42
    ...
}
```

Note: In C++ any function can use default arguments

11

DESTRUCTORS

```
class Foo{
    private:
        int data;
    public:
        Foo(): data(0) {}
        ~Foo(){}
};
```

- No return values
- Same name as the class preceded by tilde (~)
- No arguments
- The most common use of destructors is to de-allocate memory that was allocated for the object (more later)

12

Destructors

- Destructor does house-keeping when object is destroyed
- Triggered by **scope change** or **delete**
- Required when user does dynamic allocation; default is generated
- Destructor has form (when fully qualified):
ClassName::~~ClassName() {...}
- No return type or arguments: no overloading
- Should **delete** allocated memory
- Called immediately before object is destroyed

13

Conditional Operator

```

If (h >= 0 && h < 24)
    hour = h;
Else
    hour = 0;
endif

```

- `hour = ( h >= 0 && h < 24 ) ? h : 0; }`
- Condition ?
 - T – return 1st condition
 - F – return 2nd condition

14

Remainder Operator

- Cout << 6 % 8 << endl
 << 7 % 8 << endl
 << 8 % 8 << endl
 << 9 % 8 << endl
 << 10 % 8 << endl;

15

Increment Operator

- Count = count + 1;
- Count += 1;
- Count ++

16

Prefix and Postfix

- `TotalWeight = avgWeight * ++count;`

17

```
int main()
{
    int count = 10;

    cout << "count = " << count << endl;
    cout << "count = " << ++count << endl;
    cout << "count = " << count << endl;
    cout << "count = " << count++ << endl;
    cout << "count = " << count << endl;
}
```

18